LoadLead — Platform E2E Audit v2 (deep seams + recommendations)

Date: 2026-07-03 **Author:** Platform Engineering **Baseline:** `main @ 014bb87` (post-H1 merge) **Focus:** the seams v1 *sampled but did not trace* — payments ledger, settlement take-rate, compliance intercept/suppression — hunting the cross-table / read-then-write non-atomicity pattern (rec #3, #5).

---

1. Executive summary

v1 shipped: H1 (env isolation) merged, M1–M3 in a green PR, M4 (prod static keys) live. This pass traced the **money movement path end-to-end** and found **one HIGH, systemic defect the unit suites hide**: the reconciliation ledger and funding-advance service enforce idempotency with **scan-then-put** — a read-then-write race with no conditional guard. Under concurrent or retried settlement they **double-record money** (double payment-routing, double intercept application, double advance = double funding). The exact generalization of M1, now on the money ledger itself.

Everything remediated in v1 re-verified clean. 1 HIGH, 1 MEDIUM, rest green.

Severity	Finding
🔴 HIGH	**V2-H1** — money ledger + funding advances use non-atomic scan-then-put idempotency → double-record under concurrency
🟠 MEDIUM	**V2-M1** — reconciliation / funding / intercept services full-table scan on every op (scale), same class as M3
🟢 GREEN	H1 parity fixed (gate green), prod hardened, negotiation + money-primitive core sound, compliance audit log correctly append-only, suite 620/624 (the 4 = M2, pending in PR #23)

---

2. Status of v1's five recommendations

#	Recommendation	Status
1	Stand up real staging	**Partly** — 50 isolated staging tables live (~$0); hosted compute deferred on cost
2	IaC drift as a release gate	**DONE** — `check-table-env-parity.mjs` wired into dev+staging deploys; **green on main**
3	Cross-table transactional integrity at the assignment chokepoint; audit the other seams	**DONE (this pass)** — M1 fixed the assignment seam; tracing the *others* surfaced **V2-H1** (same class, money ledger)
4	Security-control smoke test (ADMIN login w/o MFA refused) vs a real env	**Open** — recommend adding once staging compute exists; unit coverage restored by M2 (#23)
5	Deep-dive payments ledger + settlement take-rate + compliance seams	**DONE (this pass)** — traced; V2-H1 is the result

---

3. Findings

🔴 V2-H1 — Money ledger idempotency is not concurrency-safe

What. Two money-critical, append-only services dedupe by *reading then writing*:

`services/reconciliationService.ts` — `recordOutcome()` (the ledger for **payment routing, reserve release, recourse buyback, non-recourse loss, supplemental advance, intercept application, adjudication compensation**):

```
if (input.idempotencyKey) {
  const existing = (await this.scanAll()).find((o) => o.idempotencyKey === input.idempotencyKey);
  if (existing) return existing;
}
const outcome = { outcomeId: Helpers.generateId('recon'), ... };  // RANDOM pk
await Database.putItem(reconciliationOutcomesTable, outcome);       // no condition
```

`services/fundingAdvanceService.ts` — `issueAdvance()`, same shape, with the hard invariant "never double-advance a line."

Failure scenario. Two concurrent (or client-retried-in-flight) calls with the same `idempotencyKey`: both `scanAll()` see no existing row, both build a row with a fresh random PK, both `putItem` → **two ledger entries / two advances for the same event**. On the funding path that is **money fronted twice**; on the reconciliation path the invoice's outcome history double-counts (double payment-routing, double intercept). It is safe under *serial* retry (the second read finds the first) — only concurrency/overlap breaks it.

Why it matters + why tests miss it. This is the settlement money path (compliance intercepts route through `recordOutcome`). Unit tests call these serially, so the race never manifests → false green. The codebase already uses the correct primitive elsewhere: **11 services** (negotiation, attestation, factoring, liquidity, verification, …) use `attribute_not_exists` / `ConditionExpression`. These two money services are the outliers.

COA (the fix). Make the idempotent write atomic — deterministic id + conditional put (mirrors `negotiationService`):

```
const outcomeId = input.idempotencyKey ? `recon_${input.idempotencyKey}` : Helpers.generateId('recon');
try {
  await docClient.send(new PutCommand({
    TableName: reconciliationOutcomesTable, Item: { ...outcome, outcomeId },
    ConditionExpression: 'attribute_not_exists(outcomeId)',
  }));
} catch (e) {
  if (isConditionFailure(e)) return (await Database.getItem(reconciliationOutcomesTable, { outcomeId }))!; // a concurr
  throw e;
}
```

Same for `fundingAdvanceService` (deterministic id from `advanceKey`). `Database.putItem` has no condition param, so use `docClient` directly (as the negotiation service does) or extend `putItem` with an optional `ConditionExpression`. Add a test that asserts two overlapping calls with the same key yield exactly one row.

🟠 V2-M1 — Reconciliation / funding / intercept reads are full-table scans

`reconciliationService`, `fundingAdvanceService`, `payoutInterceptService` all `scanAll()` on every create/list/apply (`outcomesForInvoice`, `listForInvoice`, `activeFor`, and the idempotency read above). Same O(table) scale cliff as M3, now on the money tables (which grow forever — append-only). Fine at beta volume; add an `invoiceId` (and `idempotencyKey`) GSI and query, in the same motion as the V2-H1 fix.

---

4. Re-verified GREEN (v1 remediations hold)

- **H1 env parity** — `check-table-env-parity.mjs` **green** on `main`; 50 staging tables live + isolated.
- **Prod** — `/api/health` → `productionHardened:true`; hardening intact.
- **Compliance audit log** (`adminAuditService`) — append-only `putItem`, no dedup by design → **correct**, not affected by V2-H1.
- **Negotiation + money primitives** — conditional writes + integer cents, unchanged, sound.
- **Suite** — 620/624 on `main`; the 4 failures are exactly M2 (adminMfa ×3 + errorHandler ×1), fixed in **PR #23**, not regressions.

---

5. Courses of action — prioritized

Pri	Action	Finding
P0	Merge **#23** (M1–M3) — closes the assignment-seam fix + restores the 4 tests to green	—
P1	Fix **V2-H1**: deterministic id + conditional put in `reconciliationService.recordOutcome` and `fundingAdvanceService.issueAdvance`; test the concurrent-duplicate case	V2-H1
P1	Add the **ADMIN-MFA smoke test** (rec #4) — assert login without enrolled MFA is refused	rec #4
P2	Add `invoiceId` / `idempotencyKey` GSIs to the money tables; convert scans to queries (bundle with V2-H1)	V2-M1, M3
P2	Add the two negotiation GSIs' equivalent for the money tables to the tableset module	V2-M1
P3	Stand up hosted staging so the security smoke test (rec #4) runs against a real env, not mocks	rec #1/#4

---

6. Recommendation (strategic)

The recurring theme across M1 and V2-H1 is the same: **append-only + "idempotent by reading first" is not idempotent under concurrency.** Adopt one house rule — *every idempotent write derives a deterministic key and uses a conditional put* — and sweep the money/compliance services to it (reconciliation, funding, and any future settlement seam). Pair it with the GSIs so the same change removes the scan. That single pattern closes the whole class this audit keeps surfacing.

Appendix — all findings reproducible from the audit session; every prod interaction was read-only.